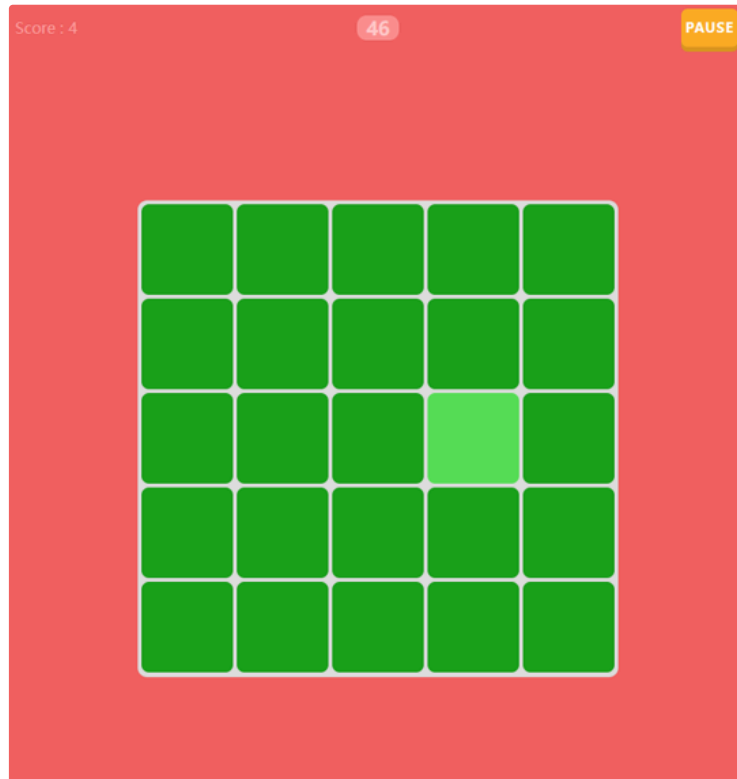


Interview task 2026

Interview Task 2026 — Frontend (Angular)

Summary

Your task is to build a small "spot-the-odd-color" game (inspired by [Kuku Kube](#)).



You start from a provided project, which contains the menu screen. You will build the gameplay and the highscores functionality.

The game is about finding an odd colored box in a grid with increasing difficulty. The grid gets bigger and the color difference gets less noticeable with each level. The goal is to complete all 5 levels in the shortest time possible. Every level is different: the color of the boxes and the position of the odd box is different for each game and each level.

The players can submit their results at the end of the game, and check the all time highscores.

What we provide

- The starting project. It's an Angular 21 project that contains the implementation for the menu, utility functions for creating esthetic colors, but you will have to create the rest.
- A REST API that you need to use for managing the highscores (see `INSTRUCTIONS.md`).

Functional requirements

1. **Menu:** In the menu clicking “Play” should navigate the user to `/game` and clicking “Highscores” should navigate the user to `/highscores`.
2. **Gameplay:** On the `/game` page, implement the game.
 - a. The grid should have colored boxes. The color of the boxes should be different for each level and game (see `box-color.utils.ts`).
 - b. All the boxes should have the same color, except the odd box that the player needs to find, which should be brighter. The position of this box should be different each level and game.
 - c. Clicking the odd box should bring you to the next level, while clicking the other boxes does nothing. The next level should have new colors and a new position for the odd box.
 - d. Each level should increase the difficulty:
 - i. grid size growing 6×6 (1st level) $\rightarrow 7 \times 7 \rightarrow \dots \rightarrow 10 \times 10$ (5th level)
 - ii. decreasing the brightness difference for the odd box: start 30% brighter, and decrease it by 5% for each level
 - e. When the game begins, start a count-up timer, and display it above the grid in `mm:ss` format.
 - f. The game should consist of 5 levels. After completing the 5th level, the game finishes.
3. **Score submission:** When the game finishes, show a form where the player can enter their name and submit their score using the provided API. This requirement is intentionally high level. The goal is to create a solution that matches the complexity of the application and provides a decent user experience.
4. **Highscores:** On the `/highscores` page, display the saved highscores in a list for the top 10 quickest players.

Technical requirements

- Usage of modern Angular features, like:
 - standalone elements (no modules)
 - new control flow syntax: `@if` / `@for` / `@switch`
 - `inject()` for dependency injection
 - functional `input()` / `output()`
 - Usage of Signals and RxJS where appropriate
- State management approach of your choosing: here the goal is not to use certain libraries, but to present the understanding of state handling
- OnPush change detection in components
- API access via `HttpClient`

- Implement a custom pipe somewhere in the app. *Hint: formatting the timer display (e.g. `mm:ss`) is a natural fit.*
- At least a very basic styling. If you want, you can use existing UI libs or CSS frameworks.

Optional

- A functionality covered by integration tests.
- A well made UI and UX.

Submission and evaluation

- Provide a short **README.md** containing a short description on your key decisions.
- Remove the `dist` and `node_modules` folders and zip the project before submitting.
- The app should run on a machine that has `Node 24` installed with `npm install` and `npm start` executed in the root folder of the project!
- We'll review your submission. If it stands out, we'll invite you for our technical interview where we will also discuss your submitted work details.

Other remarks

- There is no time limit for the task, but we assume it will take you around 2-3 hours.
- AI usage is allowed, but we expect you to have a level of understanding that you would have with manually-written code. If you use AI, submit your chat log export in the `ai-logs` project folder.
- Anything that's not described in the requirements is up to you to decide. This also means, you can use arbitrary NPM dependencies if you feel the need.